ChatGPT said:

Authentication & Authorization Service — C++17 Enterprise Project Architecture

Build this as a **standalone REST API authentication service**, similar in purpose to a Java Spring Boot application using Spring Security, PostgreSQL, and Redis.

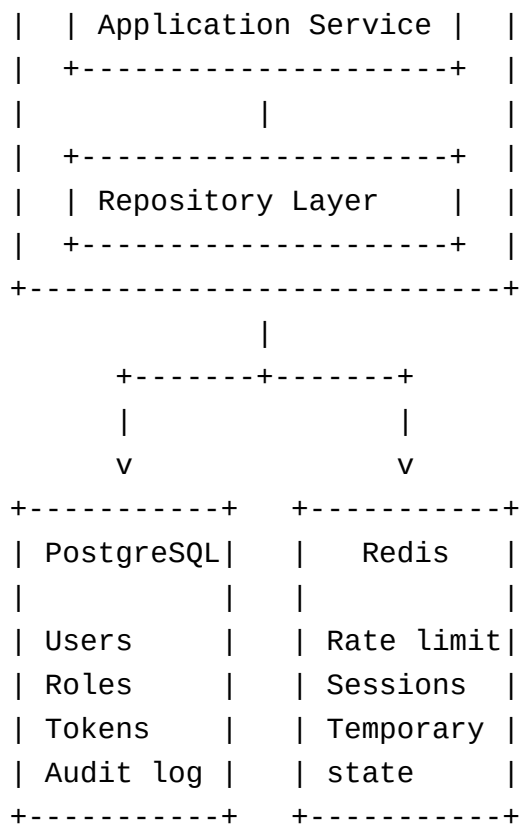
The service will handle user registration, login, authentication, role-based access control (RBAC), token validation, refresh tokens, and rate limiting.

Recommended stack: C++17 + Drogon + PostgreSQL + Redis + JWT + NGINX + CMake.

1. System Architecture




```
                CLIENT / POSTMAN
                       |
                       | HTTPS
                       v
                 +-----------+
                 |   NGINX   |
                 | TLS / Proxy
                 +-----------+
                       |
                       v
        +----------------------------+
        |   C++17 AUTH SERVICE       |
        |      Drogon Framework      |
        |                            |
        |  +---------------------+   |
        |  | Authentication API  |   |
        |  +---------------------+   |
        |             |              |
        |  +---------------------+   |
        |  | Authorization       |   |
        |  | Middleware / RBAC   |   |
        |  +---------------------+   |
        |             |              |
        |  +---------------------+   |
```



Component responsibilities

Component	Responsibility
NGINX	HTTPS termination, reverse proxy, request-size limits
Drogon	HTTP server, routing, request handling
Authentication middleware	Validates access tokens
Authorization middleware	Enforces role and permission requirements
AuthService	Registration, login, logout, token lifecycle
UserService	User account and status management
TokenService	Access-token creation and validation
PasswordService	Secure password hashing and verification
UserRepository	Reads and writes user records
TokenRepository	Persists refresh-token records

Component	Responsibility
RedisCache	Rate limiting, temporary session state, revocation state if required
PostgreSQL	Durable users, roles, refresh-token records, and audit records

Keep authentication and authorization separate:

- **Authentication:** Who are you?
- **Authorization:** What are you allowed to do?

---

2. Recommended C++17 Project Structure

Use a layered architecture. Keep business rules separate from HTTP handling, database access, and Redis.

```
AuthService/
│
├── CMakeLists.txt
├── config.json
├── Dockerfile
├── docker-compose.yml
├── .env
│
├── migrations/
│   ├── 001_create_users.sql
│   ├── 002_create_roles.sql
│   ├── 003_create_refresh_tokens.sql
│   └── 004_create_audit_logs.sql
│
├── include/
│   ├── controllers/
│   │   ├── AuthController.h
│   │   ├── UserController.h
│   │   └── AdminController.h
│   │
│   ├── services/
│   │   ├── AuthService.h
│   │   ├── UserService.h
│   │   ├── TokenService.h
│   │   └── PasswordService.h
│   │
```

```
│   ├── repositories/
│   │   ├── UserRepository.h
│   │   ├── RoleRepository.h
│   │   └── TokenRepository.h
│   │
│   ├── middleware/
│   │   ├── AuthenticationMiddleware.h
│   │   └── AuthorizationMiddleware.h
│   │
│   ├── models/
│   │   ├── User.h
│   │   ├── Role.h
│   │   └── RefreshToken.h
│   │
│   ├── cache/
│   │   ├── RedisClient.h
│   │   └── RateLimiter.h
│   │
│   └── common/
│       ├── ErrorResponse.h
│       └── AppConfig.h
│
├── src/
│   ├── main.cpp
│   ├── controllers/
│   ├── services/
│   ├── repositories/
│   ├── middleware/
│   ├── cache/
│   └── common/
│
└── tests/
    ├── AuthServiceTest.cpp
    ├── TokenServiceTest.cpp
    ├── UserRepositoryTest.cpp
    └── AuthApiTest.cpp
```

Design rule: Controllers should not execute SQL directly. They call services, and services use repositories to access persistent data.

For example:

```
AuthController
      |
      v
AuthService
```

```
|
+------> PasswordService
|
+------> TokenService
|
+------> UserRepository
|
+------> TokenRepository
|
+------> RedisClient
```

You can use Drogon's ORM for PostgreSQL and a Redis client for caching and rate limiting.

Official documentation:

- Drogon Framework
- PostgreSQL
- Redis

---

3. REST API Endpoints

Design the service around the following endpoints.

A. Authentication endpoints

Method	Endpoint	Purpose	Access
POST	`/api/v1/auth/register`	Register a new user	Public
POST	`/api/v1/auth/login`	Authenticate credentials	Public
POST	`/api/v1/auth/refresh`	Obtain a new access token	Valid refresh token
POST	`/api/v1/auth/logout`	Revoke the refresh-token session	Authenticated
POST	`/api/v1/auth/logout-all`	Revoke all user sessions	Authenticated
GET	`/api/v1/auth/me`	Get current user information	Authenticated

B. User endpoints

Method	Endpoint	Purpose	Access
GET	`/api/v1/users/me`	View own profile	Authenticated

Method	Endpoint	Purpose	Access
PATCH	`/api/v1/users/me`	Update own profile	Authenticated
PATCH	`/api/v1/users/me/password`	Change password	Authenticated
GET	`/api/v1/users`	List users	Admin
GET	`/api/v1/users/{id}`	View a user	Admin
PATCH	`/api/v1/users/{id}/status`	Enable or disable a user	Admin

C. Administrative endpoints

Method	Endpoint	Purpose	Access
GET	`/api/v1/admin/dashboard`	View administrative information	Admin
GET	`/api/v1/admin/roles`	List roles	Admin
POST	`/api/v1/admin/users/{id}/roles`	Assign a role	Admin

Important: Public registration must never allow a caller to assign themselves the `ADMIN` role. Administrative role assignments must be authorized and validated by the server.

---

4. PostgreSQL Database Design

Use PostgreSQL as the source of truth for user identities, roles, and refresh-token records.

Table 1: `users`

Column	Type	Purpose
id	UUID	Primary key
email	VARCHAR	Unique login identifier
password_hash	TEXT	Password hash
status	VARCHAR	ACTIVE, DISABLED, etc.
created_at	TIMESTAMPTZ	Registration time

Column	Type	Purpose
updated_at	TIMESTAMPTZ	Last update

Do not store plaintext passwords.

Use a password-hashing algorithm designed for passwords, such as Argon2id, with appropriately configured parameters.

Table 2: `roles`

Column	Type	Purpose
id	BIGINT	Primary key
name	VARCHAR	USER, ADMIN
description	TEXT	Role description

Table 3: `user_roles`

Column	Type	Purpose
user_id	UUID	References `users.id`
role_id	BIGINT	References `roles.id`

Use a composite primary key on `(user_id, role_id)`.

Table 4: `refresh_tokens`

Column	Type	Purpose
id	UUID	Token record ID
user_id	UUID	Token owner
token_hash	TEXT	Hash of the refresh token
expires_at	TIMESTAMPTZ	Expiration
revoked_at	TIMESTAMPTZ	Revocation time
created_at	TIMESTAMPTZ	Creation time

Store a hash of the refresh token rather than the raw token. Implement token rotation so a refresh token is replaced when used, and detect reuse of previously rotated tokens.

Table 5: `audit_logs`

Column	Type	Purpose
id	BIGINT	Primary key
user_id	UUID, nullable	Associated user
event_type	VARCHAR	LOGIN, LOGOUT, etc.
created_at	TIMESTAMPTZ	Event time
metadata	JSONB	Relevant event details

Avoid logging passwords, access tokens, refresh tokens, or other secrets.

---

5. Redis Design

Redis should accelerate selected operations rather than replace PostgreSQL.

Redis key pattern	Purpose	Expiration
`auth:login_attempts:{identifier}`	Track failed login attempts	Short TTL
`auth:rate:{identifier}`	Rate-limit requests	Short TTL
`auth:session:{session_id}`	Optional session state	Session TTL
`auth:revoked:{token_id}`	Optional access-token revocation	Until token expiration
`auth:user_roles:{user_id}`	Optional cached roles	Short TTL or explicit invalidation

The identifier used for rate limiting may combine a normalized account identifier and IP address. Avoid exposing email addresses or other personal information directly in Redis key names.

What belongs where?

PostgreSQL

- User accounts.
- Password hashes.

- Roles and role assignments.
- Refresh-token records.
- Audit events.

Redis

- Rate-limit counters.
- Optional session state.
- Short-lived caches.
- Optional token-revocation state.

For a JWT-based design, access tokens can be validated locally without querying Redis on every request. If you require immediate revocation, use a revocation mechanism such as a token denylist or session-version check.

---

6. Authentication Flow

Step 1: User registration

```
POST /api/v1/auth/register
            |
            v
      AuthController
            |
            v
       AuthService
            |
            v
      PasswordService
            |
            v
      Hash password
            |
            v
    UserRepository
            |
            v
       PostgreSQL
```

The service validates the input, hashes the password, and creates the user in a database transaction.

Step 2: User login

```
POST /api/v1/auth/login
            |
            v
        AuthService
            |
            +-----> Redis: rate-limit check
            |
            v
      UserRepository
            |
            v
       PostgreSQL
            |
            v
      PasswordService
            |
            v
    Verify password hash
            |
            v
       TokenService
            |
            +-----> Generate short-lived JWT
            |
            +-----> Create refresh token
            |
            v
      PostgreSQL + Redis
            |
            v
    Return tokens to client
```

The refresh-token record is persisted in PostgreSQL. Redis is used only for the applicable rate-limiting or temporary-state operations.

Step 3: Access a protected resource

The client sends:

```
GET /api/v1/users/me
Authorization: Bearer <access_token>
```

The authentication middleware validates the JWT signature, expiration, issuer, audience, and other required claims.

It then checks the user's status and any additional revocation or session state required by your design.

The authorization layer verifies that the user has permission to perform the requested action.

---

7. JWT Access Token vs Refresh Token

Feature	Access token	Refresh token
Purpose	Access protected APIs	Obtain a new access token
Lifetime	Short, e.g. 5–15 minutes	Longer, based on session policy
Format	Signed JWT	Prefer an opaque, random token
Storage	Client-side, handled securely	Client-side, handled securely
Server-side record	Optional, depending on revocation requirements	Persist a hashed record in PostgreSQL
Sent to	Protected API endpoints	Refresh endpoint only

JWTs should be signed with a well-maintained cryptographic library. Validate the permitted algorithm and all relevant claims; never trust an algorithm selected by an untrusted token.

For production, also consider TLS, secret management, key rotation, account lockout policies, CSRF protections where applicable, and secure client-side token storage.

---

8. C++17 Library Choices

Requirement	Recommended option
REST API	Drogon
Database	PostgreSQL
PostgreSQL ORM	Drogon ORM
Redis client	redis-plus-plus or hiredis
JSON	Drogon JSON support

Requirement	Recommended option
JWT	A maintained JWT library, such as jwt-cpp
Password hashing	Argon2id via a maintained library
Logging	spdlog
Testing	GoogleTest
Build	CMake + Ninja
Reverse proxy	NGINX
Deployment	Docker + AWS

Before adopting any security-sensitive library, verify its maintenance status, supported algorithms, compatibility with C++17, and security advisories.

---

9. Implementation Roadmap

Build the service incrementally rather than implementing every component at once.

Phase	Deliverable	Main concepts
1	Drogon server with `/health`	Routing, configuration
2	PostgreSQL connection	Database access, migrations
3	User registration	Validation, password hashing
4	Login	Credential verification
5	JWT authentication	Token generation and verification
6	Protected endpoints	Authentication middleware
7	Role-based authorization	Roles, permissions
8	Refresh-token lifecycle	Rotation, revocation, sessions
9	Redis integration	Rate limiting, caching
10	Automated tests	Unit, integration, security tests

Phase	Deliverable	Main concepts
11	NGINX and Docker	TLS, deployment
12	AWS deployment	Configuration, secrets, monitoring

Final recommendation

Start with **Phase 1: Drogon + PostgreSQL + user registration + login**. Add JWT authentication next, then RBAC, refresh-token rotation, and Redis rate limiting.

This gives you a realistic C++17 enterprise backend with a clear separation between controllers, business services, repositories, security components, and infrastructure.

If you want, I can:

- ↳ Describe how to implement refresh-token rotation securely in this architecture
- ↳ Outline the process for enforcing role-based access control in middleware
- ↳ Explain how rate limiting integrates with user login flow using Redis